AWS Cloud Infrastructure Cost Optimization Report

Comparative Analysis of Hosting Scenarios & Recommendation for URL Shortener Website

Prepared For: Prashant Bhushan	**Date:** August 7, 2026
Active Credit Balance: $88.12 USD	**Credit Days Remaining:** 172 Days
Current Burning Rate: ~$35.37/month	**Target Optimization Rate:** ~$18.27/month

Executive Summary

This report evaluates various AWS infrastructure hosting configurations for the URL Shortener application. The goal is to maximize the utility of your remaining **$88.12 AWS credits** (valid for 172 days). Currently, your single-node custom Kubernetes setup on an EC2 instance consumes **~$35.37/month**, meaning credits will run dry in **~2.5 months** (long before the 172-day expiration). By transitioning to an **Optimized Hybrid architecture**, we can reduce monthly charges to **~$18.27/month**, successfully extending your credit lifespan to **4.8 months (144 days)**.

AWS Infrastructure Scenarios Evaluated

Scenario 1: EC2 + Custom Kubernetes (Current Setup)

You currently host the entire stack (Frontend, Backend, Worker, Postgres, Redis) on a single EC2 t3.small instance running a self-managed Kubernetes control plane. Traffic is routed via an Application Load Balancer (ALB).

- **Cost:** ~$35.37/month (ALB and Public IP account for 50% of the cost).
- **Pros:** Already running, zero refactoring, robust local architecture.
- **Cons:** High overhead for small traffic; ALB ($14/mo) is highly redundant for a single VM node.

Scenario 2: Fully Serverless (Lambda + S3 + RDS + ElastiCache)

Rebuilding the application to use event-driven serverless services. The backend and worker run on AWS Lambda, the frontend resides on S3/CloudFront, and database/caching are moved to Amazon RDS and ElastiCache.

- **Cost:** ~$30.00 - $33.00/month (RDS Postgres and ElastiCache Redis are always-on and expensive).
- **Pros:** Seamless auto-scaling, zero server management, high availability.
- **Cons:** Requires heavy code refactoring (mapping APIs to API Gateway, changing background worker from loops to SQS), suffers from Lambda cold starts, and database hosting is still expensive.

Scenario 3: Managed EKS + Managed DBs

A standard enterprise Kubernetes deployment using AWS Elastic Kubernetes Service (EKS) for container orchestration, with Postgres and Redis running on managed RDS and ElastiCache.

- **Cost:** ~$131.37/month (EKS Control Plane itself costs $73/mo).
- **Pros:** Offloads API Server/etcd memory consumption from your nodes, extremely reliable.
- **Cons:** Prohibitively expensive. Your remaining $88.12 credits would run out in less than 20 days.

Scenario 4: Optimized Hybrid (Recommended)

An optimized hybrid setup specifically tailored to save money. The static frontend is deployed to S3/CloudFront acting as a CDN. The backend, worker, Postgres, and Redis remain on the EC2 t3.small instance. Crucially, the **Application Load Balancer ($14/mo) is removed**, and CloudFront routes API requests directly to a lightweight Nginx reverse proxy running on the EC2 instance.

- **Cost:** ~$21.87/month (Saves ~38% compared to Scenario 1).
- **Pros:** Slashes costs, removes ALB overhead, extends credit lifespan, keeps deployment simple.
- **Cons:** Minor configuration changes to point CloudFront to the EC2 backend API.

Side-by-Side Comparison Matrix

Comparison Factor	Scenario 1 EC2 + K8s	Scenario 2 Serverless	Scenario 3 EKS Managed	Scenario 4 Optimized Hybrid
Monthly Cost	~$35.37	~$31.00	~$131.37	**~$21.87 (WINNER)**
Credit Lifespan	2.5 Months	2.8 Months	< 1 Month ■	**4.0 Months (120 days)**
Auto-Scaling	Manual / HPA limits	Instant Serverless	Excellent Elasticity	Manual / HPA limits
Cold Starts	None (Always On)	1-3 Seconds	None (Always On)	None (Always On)
Code Changes	None (Already setup)	Heavy Refactoring	Minor Adjustments	None (Infra only)
Complexity	Medium	High	High	Low-Medium

Deep Dive: Recommended Hybrid Scenario (Scenario 4)

By using S3 and CloudFront for the frontend assets, we take the heavy lifting of serving static files (HTML, CSS, JS, Images) off the EC2 instance. More importantly, we eliminate the costly Application Load Balancer (ALB) and route user requests directly to your EC2 instance via CloudFront. Here is the exact cost listing for Scenario 4:

AWS Component	Used For	Monthly Cost
S3 Bucket	Hosting Frontend Static Files (HTML/JS/CSS)	$0.02 (Est.)
CloudFront CDN	Global distribution & Routing HTTPS to EC2	$0.50 (Est. based on bandwidth)

EC2 t3.small	Hosting K8s pods (Backend, Worker, Postgres, Redis)	$15.77
EBS Volume (22GB GP3)	Persistent database disk space (Postgres/Redis data)	$2.00
AWS Public IP	Required for EC2 to be reachable by CloudFront	$3.60
Application Load Balancer (ALB)	**REMOVED (Replaced by direct Nginx/Node ingress)**	**$0.00 (Saved $14.00!)**
Total Monthly Cost	**Optimized Hybrid Infrastructure**	**~$21.89 / Month**

Implementation Plan (How to Migrate)

To safely transition to the Optimized Hybrid scenario, follow these three stages:

Stage 1: Build & Deploy Frontend to S3 + CloudFront

1. Build your frontend code locally or in CI (e.g. npm run build) to generate static files.
2. Create an AWS S3 bucket, enable static website hosting, and upload the build directory files.
3. Set up a CloudFront distribution pointing to the S3 bucket as the default origin.

Stage 2: Configure Routing for Backend APIs

1. Configure CloudFront with an additional Origin pointing directly to your EC2 instance's public IP address.
2. Create a Cache Behavior in CloudFront: Send any traffic matching /api/* to the EC2 Backend origin (bypassing S3 caching).
3. Configure NodePort or a HostPort ingress controller on your EC2 single-node Kubernetes cluster to accept incoming traffic on port 80/443 without an ALB.

Stage 3: Remove ALB and Clean Up

1. Point your domain name (e.g. via Route53) to the CloudFront distribution instead of the ALB.
2. Test both frontend and backend functionality thoroughly to verify API communication works via CloudFront.
3. Delete the Application Load Balancer (ALB) from your AWS EC2 Console to stop the $14.00/month recurring charge.